Algoritmos optimizadores

Natalia Giraldo Amador*, Valerie S. Olave Pineda†
Facultad de Ingeniería, Universidad de San Buenaventura Cali, Cali, Colombia
*ngiraldoa@correo.usbcali.edu.co, †vsolavep@correo.usbcali.edu.co

I. Introducción

Los algoritmos optimizadores juegan un papel importante en el proceso de entrenamiento de los modelos de aprendizaje automático y aprendizaje profundo. Estos se encargan de ajustar iterativamente los parámetros (pesos y sesgos) internos del modelo para minimizar el error, guiandola hacia mejores predicciones mediante la minimización de la función de pérdida [1]. Existen varios algoritmos optimizadores que son variaciones del descenso del gradiente, cada uno con sus fortalezas para resolver un tipo de problema específico. La elección del optimizador no solo determina la velocidad de convergencia del modelo, sino que la elección inadecuada puede llevar a una convergencia en un mínimo local [2]. En este trabajo se evaluará el desempeño de seis algoritmos optimizadores para determinar su eficiencia en la optimización de la función de varias variables Himmelblau, una función usada frecuentemente como prueba en este campo debido a que tiene cuatro mínimos locales [3], lo que la convierte en una buena elección para analizar cada optimizador.

II. Materiales y métodos

A. Stochastic Gradient Descent (SGD)

El descenso del gradiente estocástico es un método de optimización usado generalmente cuando se trabaja en grandes conjuntos de datos. Siendo este una variante del descenso de gradiente tradicional que se basa en todo el conjunto, usa una sola muestra de datos seleccionada aleatoriamente a la vez. De este modo es menos costoso computacionalmente, mucho más rápido y tiene mayor escalabilidad. Por otro lado, puede generar un proceso de convergencia más ruidoso, resultando en no llegar al mínimo global [4], [5], [6].

El descenso del gradiente se actualiza de la siguiente forma:

$$\theta = \theta - \eta \, \nabla_\theta J(\theta;\, x_i, y_i)$$

donde:

- x_i y y_i representan los features y el target correspondientemente.
- El gradiente $\nabla_\theta J(\theta; x_i, y_i)$ ahora se calcula en pequeños bloques.
- El gradiente se calcula de forma que:

$$\nabla_\theta J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta L\Big(x^{(i)}, y^{(i)}, \theta\Big)$$

B. RMSProp (Root Mean Square Propagation)

Este algoritmo propuesto por Geoff Hinton adapta la tasa de aprendizaje utilizando un promedio exponencial de los cuadrados de los gradientes recientes en lugar de acumular todos los gradientes pasados, logrando que la actualización de parámetros sea estable y el modelo pueda seguir aprendiendo en etapsas avanzadas del entrenamiento [7].

La formula de RMSProp corresponde a:

$$cache_{\text{new}} = \gamma \cdot cache_{\text{old}} + (1-\gamma) \left(\frac{\partial \, Loss}{\partial W_{\text{old}}} \right)^2$$

donde:

- $cache_{\text{new}}$ representa el nuevo valor del promedio exponencial de los cuadrados de los gradientes.
- $cache_{\text{old}}$ es el valor anterior del promedio exponencial de los cuadrados de los gradientes, que representa la información acumulada en las iteraciones previas.
- γ (gamma): es la tasa de olvido del promedio exponencial, esta generalmente esta cerca a 0.9.
- $1-\gamma$: es el peso que se le da al gradiente actual en el promedio exponencial.
- $Loss$: funcion de costo, mide el error del modelo.
- W_{old}: son los pesos actuales del modelo antes de ser actualizados.
- $\left(\frac{\partial \, Loss}{\partial W_{\text{old}}}\right)^2$: es el cuadrado del gradiente de la funcion de perdida respecto a los pesos, utilizado para calcular la magnitud del camvio y crear el promedio de gradientes cuadrados.

La regla de actualizacion es:

$$\theta = \theta - \frac{\eta}{\sqrt{v_t + \epsilon}} \cdot g_t$$

C. Adaptive Moment Estimation (Adam)

El optimizador Adam es un algoritmo de optimización iterativo, este es una combinación de RMSprop y SGD con momentum para ajustar la tasa de aprendizaje durante el entrenamiento. Es eficaz con grandes conjuntos de datos y modelos complejos debido a que adapta la tasa de aprendizaje para cada parámetro automáticamente, este evita la convergencia lenta y la inestabilidad mientras llega al mínimo global [8], [9]. Funciona de modo que:

- **Estimación del primer momento (media):**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \frac{\partial L}{\partial w_t}$$

- **Estimación del segundo momento (Varianza):**

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left(\frac{\partial L}{\partial w_t}\right)^2$$

- **Corrección del bias:** Debido a que m_t y v_t están inicializados en cero, tienden a estar sesgados hacia cero. Por lo que se calcula la estimación de la siguiente manera:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

- **Actualización final de pesos:**

$$w_{t+1} = w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}\,\alpha$$

Donde:

- α: Es la tasa de aprendizaje
- β_1 y β_2: Tasa de olvido para las medias móviles del gradiente y el gradiente cuadrado
- ϵ: Una constante pequeña positiva para evitar la división por cero.

D. Adadelta

Este algoritmo ajusta la velocidad de aprendizaje de cada parametro automaticamente durante el entrenamiento sin fijar manualmente un valor global, usando promedios moviles de las magnitudes de los gradientes y de las actualizaciones anteriores para escalar los pasos [10], [11]. La formula correspondiente es:

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2$$

Donde:

- s_t: es el promedio exponencial del cuadrado de los gradientes.
- s_{t-1}: valor anterior al promedio.
- ρ: tasa de olvido.
- g_t: gradiente actual.
- g_t^2: cuadrado del gradiente.

Actualización del parámetro:

$$X_t = X_{t-1} - g_t'$$

- x_t: nuevo valor del parámetro.
- x_{t-1}: valor anterior del parámetro.
- g_t': gradiente reescalado.

$$g_t' = \frac{\sqrt{\Delta X_{t-1} + \epsilon}}{\sqrt{s_t + \epsilon}} \odot g_t \qquad \Delta X_t = \rho\, \Delta X_{t-1} + (1-\rho)\, g_t'^2$$

- g_t' es el gradiente ajustado o reescalado.
- g_t es el gradiente original en la iteración t.
- s_t es el promedio exponencial de los gradientes al cuadrado.
- Δx_{t-1} es el promedio exponencial de las actualizaciones anteriores al cuadrado.
- ϵ es una constante pequeña que evita divisiones por cero.
- $\odot$ representa la multiplicación elemento a elemento.

E. Adaptive Gradient Algorithm (Adagrad)

Este algoritmo ajusta la tasa de aprendizaje de cada parametro durante el entrenamiento usando gradientes pasados para escalar las actualizaciones, haciendolo efectivo para datos dispersos o modelos que no requieren una tasa de convergencia muy rápida pero si un proceso de optimización más estable [12], [13]. Funciona de la siguiente manera:

- **Inicialización:** Comienza inicializando los valores de los parametros aleatoriamente, además que inicializa una suma continua de gradientes al cuadrado para cada parámetro.
- **Cálculo del gradiente:** Para cada paso de entrenamiento se calcula el gradiente de la función de pérdida respecto a los parametros del modelo.
- **Tasa de aprendizaje adaptativa:** En lugar de usar una tasa de aprendizaje fija, se ajusta la tasas de aprendizaje de cada parámetro en función de la suma acumulada de gradientes al cuadrado.

De modo que para calcular la tasa de aprendizaje por cada parámetro se calcula de la siguiente manera:

$$lr_t = \frac{\eta}{\sqrt{G_t + \epsilon}}$$

Donde:

- η: Es la tasa de aprendizaje global
- G_t: Es la suma de gradientes al cuadrado para un párametro dado hasta paso de tiempo t
- ϵ: Es un pequeño valor agregado para evitar la división por cero.

Los parámetros del modelo se actualizan restando el producto de la tasa de aprendizaje adaptativo y el gradiente:

$$\theta_{t+1} = \theta_t - lr_t\, \nabla_\theta$$

Donde:

- θ_t: es el parámetro actual
- $\nabla_\theta J(\theta)$: Es el gradiente de la función de pérdida respecto al parámetro

F. Nesterov Adaptive Moment Estimation (Nadam)

Este algoritmo es una combinación de Adam haciendo uso de su tasa de aprendizaje adaptativo junto con el momento de Nesterov, que mira hacia adelante para ajustar la dirección del gradiente de forma más eficaz. De este modo Nadam suele mejorar la estabilidad del entrenamiento y la velocidad de convergencia. Se usa especialmente para redes neuronales profundas con gradientes dispersos. Funciona de modo que calcula la media y el cuadrado de los gradientes como Adam, pero aplica la anticipación de Nesterov [14], [15]:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\beta_1 \hat{m}_t + \dfrac{1-\beta_1}{1-\beta_1^t}\, g_t}{\sqrt{\hat{v}_t + \epsilon}} \right)$$

III. Resultados

Se implementaron los optimizadores sobre la función de varias variables $f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$

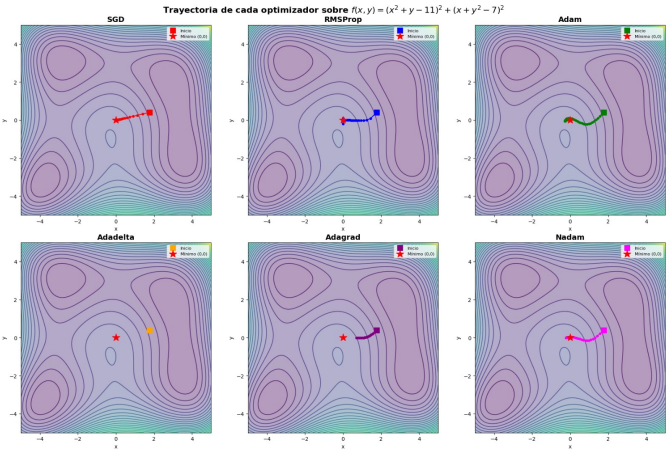

Fig. 1: Comparación visual de el paso de los optimizadores para encontrar el mínimo global de una función.

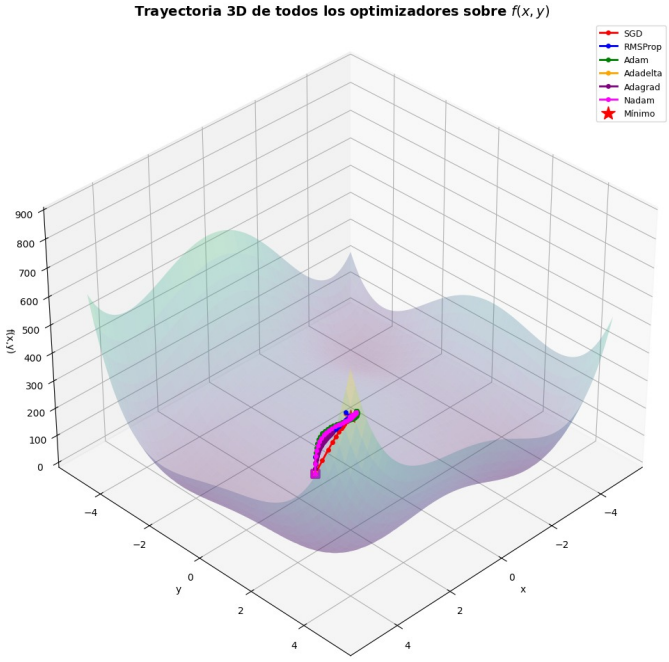

Fig. 2: Trayectoria de los optimizadores en una gráfica 3d

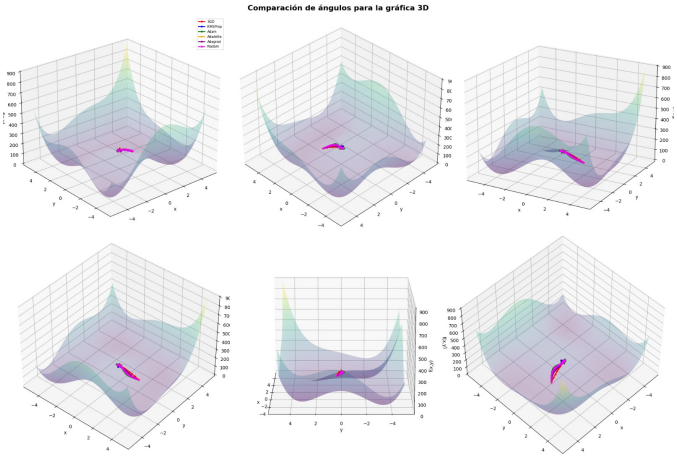

Fig. 3: Trayectoria de los optimizadores en una gráfica 3d visto desde distintos ángulos

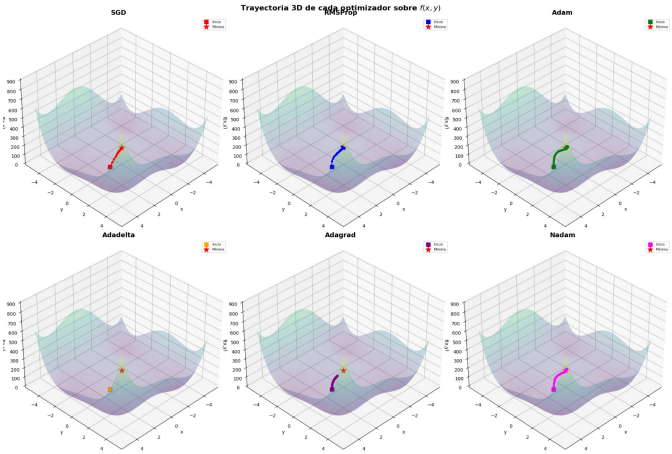

Fig. 4: Trayectoria de los optimizadores en una gráfica 3d individualmente

IV. Discusión

Se graficó el Error cuadrático medio obtenido por todos los optimizadores evaluados y como se evidencia en la figura 4, exsiten diferencias en el desempeño de cada optimmizador para la función analizada.

El oprimizador que presenta el peor rendimiento corresponde a Adadelta obteniendo un MSE=3.21. Este resultado indica que el algoritmo no logró acercarse al mínimo global de la función en un número de épocas considerable. Se considera como posible causa la forma de ajuste de las actualizaciones que utiliza el optimizador, pudiendo causar que la convergencia sea más lenta.

Por su parte, Adagrad muestra una mejoría alcanzando un error de MSE=0.49. Esto indica que el optimizador logró acercarse de manera más efectiva al mínimo de la función adaptando la tasa de aprendizaje en función de la acumulación de los gradientes pasados, logrando una convergencia más estable en comparación con Adadelta en este caso de estudio.

Finalmente, el mejor desempeño alcanzado lo obtiene el optimizador Stochastic Gradient Descend alcanzando un error de MSE=0.00. Demostrando que el algoritmo logró encontrar la solución más óptima para la función evaluada, convergiendo de manera precisa en función al mínimo.

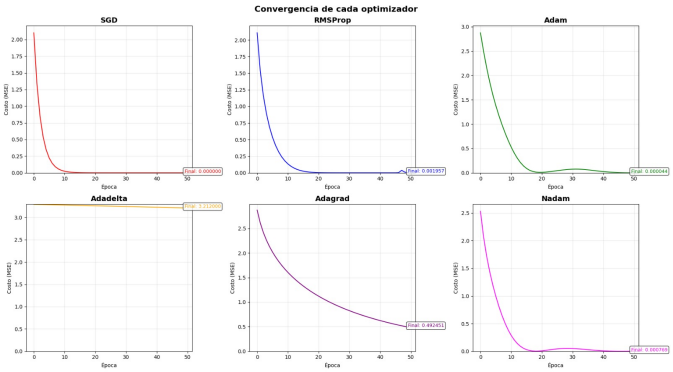

Fig. 5: Gráfica comparativa del error cuadrático medio en optimizadores.

V. Conclusiones

Los optimizadores son herramientas útiles para potenciar la efectivad de los modelos de aprendizaje profundo, ya que permiten ajustar los parámetros del modelo de forma iterativa teniendo como objetivo minimizar la función de pérdida. En este trabajo se estudiaron y evaluaron diferentes optimizadores aplicados a la función de Himmelblau, una función frecuentemente utilizada en problemas de optimización debido a la presencia de múltiples puntos mínimos.

En este trabajo fue posible evidenciar las diferencias significativas en la convergencia de cada uno de los optimizadores evaluados. En algoritmos adaptativos como Adagrad y Adadelta que ajustan dinámicamente la tasa de aprendizaje en base a información acumulada de los gradientes, se evidenció que esto puede ocasionar que las actualizaciones sean demasiado pequeñas al pasar las iteraciones, afectando la velocidad de la convergencia o en general la capacidad de alcanzar el mínimo global de la función evaluada.

En contraste, el algoritmo Stochastic Gradient Descend (SGD) mostró un desempeño significativamente superior en este estudio, logrando minimizar la función con un error cuadrático a medio cercano a cero. Demostrando que en ciertos escenarios y dependiendo de la función objetivo, métodos de optimización más simples resultan altamente efectivos.

Este trbajo demuestra que la elección del optimizador juega un rol escencial en el proceso de entrenamiento de modelos de aprendizaje profundo y de aprendizaje automático. Debido a que cada algoritmo presenta ventajas y limitaciones dependiendo del problema al que se vean aplicados y los parámetros utilizados. Por esta razón la elección adecuada y el correcto ajuste de sus parámetros garantizan un proceso de aprendizaje más eficiente y estable.

References

[1] CCIP – Nekomath, "Optimization algorithms," https://ccip.nekomath.com/CC13.html, accessed: 2026-03-06.

[2] Ultralytics, "Optimization algorithm: Ai training guide," https://www.ultralytics.com/es/glossary/optimization-algorithm, 2024, accessed: 2026-03-06.

[3] M. A. Ardeh, "Himmelblau function," https://benchmarkfcns.info/doc/himmelblaufcn.html, 2024, accessed: 2026-03-06.

[4] GeeksforGeeks, "Stochastic gradient descent (SGD)," https://www.geeksforgeeks.org/machine-learning/ml-stochastic-gradient-descent-sgd/, 2024, accessed: 2026-03-06.

[5] IBM, "Stochastic gradient descent," https://www.ibm.com/mx-es/think/topics/stochastic-gradient-descent, 2024, accessed: 2026-03-06.

[6] CodeSignal, "Stochastic gradient descent: Theory and implementation in Python," https://codesignal.com/learn/courses/gradient-descent-building-optimization-algorithms-from-scratch/lessons/stochastic-gradient-descent-theory-and-implementation-in-python, 2024, accessed: 2026-03-06.

[7] Musstafa, "Optimizers in deep learning," https://musstafa0804.medium.com/optimizers-in-deep-learning-7bf81fed78a0, 2023, accessed: 2026-03-06.

[8] GeeksforGeeks, "Adam optimizer," https://www.geeksforgeeks.org/deep-learning/adam-optimizer/, 2024, accessed: 2026-03-06.

[9] Cornell University – Computational Optimization Open Textbook, "Adam," https://optimization.cbe.cornell.edu/index.php?title=Adam, accessed: 2026-03-06.

[10] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, "Adadelta," https://d2l.ai/chapter_optimization/adadelta.html, 2023, accessed: 2026-03-06.

[11] Cornell University – Computational Optimization Open Textbook, "Adadelta," https://optimization.cbe.cornell.edu/index.php?title=Adadelta, accessed: 2026-03-06.

[12] DataCamp, "AdaGrad optimizer explained," https://www.datacamp.com/es/tutorial/adagrad-optimizer-explained, 2024, accessed: 2026-03-06.

[13] GeeksforGeeks, "Intuition behind AdaGrad optimizer," https://www.geeksforgeeks.org/machine-learning/intuition-behind-adagrad-optimizer/, 2024, accessed: 2026-03-06.

[14] Cornell University – Computational Optimization Open Textbook, "Nadam," https://optimization.cbe.cornell.edu/index.php?title=Nadam, accessed: 2026-03-06.

[15] GeeksforGeeks, "Nesterov adaptive moment estimation (NAdam)," https://www.geeksforgeeks.org/artificial-intelligence/nesterov-adaptive-moment-estimation-nadam/, 2024, accessed: 2026-03-06.